
Bus Transit Route Network Design Problem

**Project Final Report
EMU676 Optimization Models and Algorithms in
Transportation and Distribution**

Fatih Alper Vural – “N25123161”

June 3, 2026

Hacettepe University

Contents

1	Introduction	1
2	Problem Formulation	1
2.1	Problem Description	1
2.2	Mathematical Model	2
3	Description of the Metaheuristic Algorithm	2
3.1	The Algorithm	3
3.2	Initial Candidate Route Set Generation	4
3.3	Network Analysis Procedure	4
3.4	Solution Encoding and Neighbourhood Operator	5
3.5	Constraint Handling	5
3.6	Genetic Algorithm Baseline	5
4	Results	5
4.1	Test Instance	5
4.2	Parameter Tuning	6
4.3	Computational Results	6
4.4	Best Solution Found	7
4.5	Statistical Analysis	9
5	Source Code Repository	9
6	Conclusions	10
	Bibliography	11

1 Introduction

The Bus Transit Route Network Design Problem (BTRNDP) seeks the set of bus routes and their service frequencies that best serve a given travel demand while balancing the conflicting interests of passengers and operators. The problem is NP-hard: the number of candidate route configurations grows exponentially with the size of the network, so exact methods are intractable for realistic instances and metaheuristics are the standard solution approach. This report implements and evaluates the Simulated Annealing (SA) framework of Fan and Machemehl [1], in which the problem is modelled at the distribution-node level, unsatisfied demand enters the objective directly, and route frequencies are determined endogenously through a transit-assignment model.

The remainder of this report is organised as follows. In section 2 the BTRNDP is defined and its mathematical model is stated. In section 3 the SA metaheuristic is introduced, together with the candidate route generator, the network analysis (evaluation) procedure, the solution encoding and neighbourhood operator, and the constraint-handling scheme. Section 4 describes the test instance, the parameter tuning, and the computational results, including a statistical comparison of SA against a Genetic Algorithm (GA) baseline. We conclude the report in section 6. The complete source code, input data, and result files are available in the public repository referenced in section 5.

2 Problem Formulation

2.1 Problem Description

The transportation network is modelled as a directed graph $G = (N, A)$, where N is a set of nodes (each representing a demand centroid as well as a physical junction) and A a set of directed arcs. A *route* is a sequence of nodes in which each consecutive pair is connected by an arc; the *headway* of a route is the inter-arrival time of its buses and the *frequency* is its reciprocal. Travel demand d_{ij} between centroids i and j may be served *directly* (a single route connects i and j), via a *transfer* (two routes sharing a common node), or it may be left *unsatisfied*.

The BTRNDP simultaneously minimises three competing cost components: (i) passenger user cost (in-vehicle time, waiting time, and transfer penalties); (ii) operator cost, proportional to the total vehicle-hours deployed; and (iii) unsatisfied-demand cost, which penalises O-D pairs for which no feasible path exists. Weights C_1 , C_2 , C_3 balance the three components. As Fan and Machemehl [1] observe, omitting the third component would render the trivial “no service” solution optimal, so its inclusion is essential.

2.2 Mathematical Model

The BTRNDP is formulated as a multiobjective nonlinear mixed-integer program [1]. The objective minimises the weighted sum of user cost, operator cost, and unsatisfied-demand cost:

$$\begin{aligned} \min z = & C_1 \left[\sum_{i \in N} \sum_{j \in N} \sum_{r_m \in DR_{ij}} d_{ij}^{r_m} t_{ij}^{r_m} + \sum_{i \in N} \sum_{j \in N} \sum_{tr \in TR_{ij}} d_{ij}^{tr} t_{ij}^{tr} \right] \\ & + C_2 \frac{C_v}{C_m} O_v \sum_{m=1}^M \frac{T_{r_m}}{h_{r_m}} \\ & + C_3 \frac{C_d}{C_m} \left[\sum_{i,j \in N} d_{ij} - \sum_{i,j \in N} \sum_{r_m \in DR_{ij}} d_{ij}^{r_m} - \sum_{i,j \in N} \sum_{tr \in TR_{ij}} d_{ij}^{tr} \right] \quad (1) \end{aligned}$$

subject to

$$h_{\min} \leq h_{r_m} \leq h_{\max}, \quad \forall r_m \in R \quad (2)$$

$$L_{r_m} = \frac{Q_{r_m}^{\max} h_{r_m}}{P} \leq L_{\max}, \quad \forall r_m \in R \quad (3)$$

$$\sum_{m=1}^M \frac{T_{r_m}}{h_{r_m}} \leq W \quad (4)$$

$$D_{\min} \leq D_{r_m} \leq D_{\max}, \quad \forall r_m \in R \quad (5)$$

$$M \leq R_{\max} \quad (6)$$

$$\sum_i \sum_j \left(\sum_{r_m \in DR_{ij}} d_{ij}^{r_m} + \sum_{tr \in TR_{ij}} d_{ij}^{tr} \right) \geq (1 - U_{\max}) \sum_{i,j \in N} d_{ij} \quad (7)$$

$$M, h_{r_m}, Q_{r_m}^{\max}, d_{ij}^{r_m}, d_{ij}^{tr} \in \mathbb{Z}^+. \quad (8)$$

Here M is the number of routes, r_m the m -th route, D_{r_m} its length, T_{r_m} its round-trip time, h_{r_m} its headway, L_{r_m} its load factor, and $Q_{r_m}^{\max}$ the maximum passenger flow on its critical link; DR_{ij} and TR_{ij} are the sets of direct routes and transfer paths serving (i, j) . Constraint (2) enforces policy headways; (3) prevents overloading the critical link; (4) limits deployed buses to the fleet W ; (5) bounds route length; (6) caps the number of routes; (7) guarantees the served fraction of demand; and (8) enforces integrality. The baseline weights are $(C_1, C_2, C_3) = (0.4, 0.4, 0.2)$.

3 Description of the Metaheuristic Algorithm

The solution framework comprises three integrated components: (1) an Initial Candidate Route Set Generation Procedure (ICRSGP) that precomputes a pool of feasible candidate routes; (2) a Network Analysis Procedure (NAP) that evaluates any route set by assigning demand and determining

frequencies; and (3) a Simulated Annealing (SA) loop that searches the candidate pool for the best route set. Simulated Annealing [4] is a local-search metaheuristic that escapes local optima by occasionally accepting worse solutions with a probability that decreases as a “temperature” parameter is cooled.

3.1 The Algorithm

The top-level procedure is given in Algorithm 1. An outer loop grows the route-set size n from 1 to $R_{\max}$ (line 3); for each size an inner SA search is run on solutions of that fixed size. A candidate neighbour is generated by the *swap* operator (line 8), and accepted if it improves the objective or, otherwise, with the Metropolis probability $e^{-\Delta z/T}$ (line 13). The temperature T is reduced geometrically by the factor α every K iterations (line 17), and the search stops after $G_{\max}$ cooling steps. The best feasible solution encountered across all sizes is returned.

Algorithm 1 SA Framework for the BTRNDP

```

1:  $Pool \leftarrow \text{ICRSGP}()$ 
2:  $Best \leftarrow \emptyset$ 
3: for  $n = 1$  to  $R_{\max}$  do
4:    $S \leftarrow \text{RandomSolution}(Pool, n)$ ;  $T \leftarrow T_0$ ;  $g \leftarrow 0$ ;  $k \leftarrow 0$ 
5:   while  $g < G_{\max}$  do
6:     evaluate  $z(S)$  via  $\text{NAP}(S)$ 
7:      $S' \leftarrow \text{Neighbour}(S)$ ; evaluate  $z(S')$ 
8:      $\Delta z \leftarrow z(S') - z(S)$ 
9:     if  $\Delta z < 0$  then
10:       $S \leftarrow S'$ 
11:      if  $z(S') < z(Best)$  then
12:         $Best \leftarrow S'$ 
13:      end if
14:    else
15:      draw  $p \sim U[0, 1]$ 
16:      if  $p < e^{-\Delta z/T}$  then
17:         $S \leftarrow S'$ 
18:      end if
19:    end if
20:     $k \leftarrow k + 1$ 
21:    if  $k \geq K$  then
22:       $T \leftarrow \alpha T$ ;  $k \leftarrow 0$ ;  $g \leftarrow g + 1$ 
23:    end if
24:  end while
25: end for
26: return  $Best$ 

```

3.2 Initial Candidate Route Set Generation

The ICRSGP, shown in Algorithm 2, enumerates feasible candidate routes once, before optimisation. For every centroid pair (i, j) , Dijkstra’s algorithm [2] produces the shortest path and Yen’s algorithm [3] produces up to $k = 6$ loopless alternatives. A generated path is retained only if its length satisfies $D_{\min} \leq D_{r_m} \leq D_{\max}$, so that route-length feasibility (5) holds for every pool member by construction. Each route and its reverse are canonicalised to a single representative to avoid duplicate work. On the test instance this produces a pool of 569 feasible candidate routes in 0.03 seconds.

Algorithm 2 ICRSGP

```

1:  $Pool \leftarrow \emptyset$ 
2: for every centroid pair  $(i, j)$  do
3:   compute shortest path (Dijkstra); generate  $k$  alternatives (Yen)
4:   for each generated path  $P$  do
5:     if  $D_{\min} \leq \text{length}(P) \leq D_{\max}$  then
6:       canonicalise  $P$  (merge with its reverse); add  $P$  to  $Pool$ 
7:     end if
8:   end for
9: end for
10: return  $Pool$ 

```

3.3 Network Analysis Procedure

Given a route set, the NAP returns the objective value (1) together with feasibility information. It performs a lexicographic transit assignment followed by endogenous frequency setting, iterated to convergence (three iterations in the implementation):

- *Priority 1 – direct service.* If one or more routes serve (i, j) directly, the demand d_{ij} is split among them inversely proportional to in-vehicle travel time, so that faster routes attract more passengers.
- *Priority 2 – one transfer.* If no direct route exists, demand is routed over a pair of routes sharing a common node, with a transfer penalty of five equivalent minutes added to the travel time.
- *Unsatisfied.* Demand with no path of either kind is recorded as unsatisfied and penalised through the third term of (1).

After each assignment pass, the headway of each route is recomputed from the flow $q_{\max}$ on its critical link using the load-factor relation $h \leq 60 L_{\max} P / q_{\max}$, clamped to $[h_{\min}, h_{\max}]$, and the demand is reassigned. For efficiency the assignment logic is implemented as a single reusable routine, and each route

precomputes its segment times, cumulative times, and node positions; this brings a full SA run to roughly 3.4 seconds.

3.4 Solution Encoding and Neighbourhood Operator

A solution is encoded as a *set of integer indices* into the candidate pool, of cardinality $n \leq R_{\max}$. This compact encoding makes the SA loop independent of route geometry and guarantees length feasibility by construction. The neighbourhood operator $\text{Neighbour}(S)$ performs a single-route *swap*: one index in S is replaced by a uniformly random index from the pool that is not already present. This move is inexpensive, preserves the solution size n , and induces a connected search space, which is the standard requirement for the convergence behaviour of SA.

3.5 Constraint Handling

Constraints (5) and (6) are enforced structurally by the encoding. The remaining constraints – fleet size (4), unsatisfied demand (7), and load factor (3) – are handled with a *fixed-plus-marginal* penalty added to (1). For each violated constraint a large fixed term (5000) plus a marginal term proportional to the amount of violation is added. The large fixed component guarantees that any feasible solution dominates any infeasible one, while the marginal component gives the search a usable gradient out of the infeasible region. This design proved necessary because the test demand is deliberately heavy relative to the fleet, so most random starts are infeasible.

3.6 Genetic Algorithm Baseline

For comparison, a Genetic Algorithm (GA) [5] was implemented over the same encoding. A chromosome is a set of at most $R_{\max}$ route indices; selection is by binary tournament; crossover takes the union of two parents and trims it to a feasible size; and mutation swaps one route index. The GA uses a population of 24 for 40 generations and is evaluated with the identical NAP, so that the comparison is fair.

4 Results

4.1 Test Instance

Because the exact node coordinates and demand matrix of the published Mandl-scale network in [1] are not available for verbatim reproduction, a fully-specified benchmark of comparable scale was generated deterministically (without a random seed) and is shipped with the code so that all results are exactly reproducible. The instance has 15 nodes and 23 undirected links

(46 directed arcs); travel time is 2 min/km (a 30 km/h operating speed). Demand follows a gravity model $d_{ij} = \text{round}(K w_i w_j / \text{dist}_{ij})$ with node weights w and calibration constant $K = 38$; flows below 5 pass./h are dropped. The result is 102 O–D pairs with positive demand and a total demand of 7172 pass./h. The operational parameters are summarised in Table 1.

Parameter	Value	Parameter	Value
Max. routes $R_{\max}$	8	Bus capacity P	90
Max. unsatisfied $U_{\max}$	0.15	Fleet size W	58
Route length $[D_{\min}, D_{\max}]$	[3, 30] km	Load factor $L_{\max}$	1.0
Headway $[h_{\min}, h_{\max}]$	[5, 30] min	Weights (C_1, C_2, C_3)	(0.4, 0.4, 0.2)

Table 1: Operational parameters of the test instance.

4.2 Parameter Tuning

The SA has four parameters: the initial temperature T_0 , the cooling coefficient α , the maximum number of cooling steps $G_{\max}$, and the repetition counter K . Following the tuning protocol of [1], a grid over $T_0 \in \{500, 1000, 2000, 5000\}$ and $\alpha \in \{0.5, 0.6, 0.7, 0.9\}$ was evaluated, with three replications per cell using a shortened schedule ($G_{\max} = 15$, $K = 8$). The selection criterion is the lowest mean objective combined with the smallest standard deviation. Table 2 reports the mean (and standard deviation) of z for each cell.

$T_0 \backslash \alpha$	0.5	0.6	0.7	0.9
500	50859 (490)	50859 (490)	50859 (490)	51526 (1312)
1000	50482 (588)	50795 (466)	50795 (466)	51959 (1908)
2000	50632 (323)	50462 (869)	51258 (1058)	51784 (2372)
5000	48934 (2025)	52178 (1711)	52426 (989)	50908 (1331)

Table 2: Parameter-tuning grid: mean z with standard deviation in parentheses, three runs per cell. The single lowest mean ($T_0=5000, \alpha=0.5$) is unstable ($\delta = 2025$); the most stable strong cell (in bold) is $T_0=2000, \alpha=0.6$.

The grid confirms the parameter region reported by Fan and Machemehl [1]. Although the single lowest mean occurs at $T_0=5000, \alpha=0.5$, that cell has very high variance ($\delta = 2025$). Applying the joint mean–variance criterion, the configuration $T_0=2000, \alpha=0.6$ gives the best trade-off (mean 50462, $\delta = 869$) and matches the optimal values of [1]; together with $G_{\max} = 20$ and $K = 10$ it is adopted for all final experiments.

4.3 Computational Results

Each method was run for 20 independent replications with distinct seeds. The key performance measure, following [1], is the *gap* between the best–

known solution and the mean performance of the metaheuristic,

$$gap = 100 \cdot \frac{(Mean - Best)}{Best},$$

where $Best$ is the lowest objective found across all SA and GA runs ($z^* = 45361.1$), used as the reference because no published optimum exists for the generated instance. Table 3 reports the aggregate performance and Table 4 lists the per-run objectives and gaps.

Method	Mean z	Std z	Min z	Max z	Gap (%)	Time (s)
SA	48841.7	2405.2	45361.1	52989.5	7.67	3.37
GA	52940.7	2996.2	45855.7	58148.6	16.71	2.19

Table 3: Aggregate performance over 20 runs ($T_0=2000$, $\alpha=0.6$, $G_{\max}=20$, $K=10$).

SA attains a mean objective 7.7% lower than the GA and a markedly tighter distribution. Of the 20 SA runs, 8 finished within 5% of the best-known solution, against only 1 for the GA. The higher mean CPU time of SA (3.37 s versus 2.19 s) reflects the iterative frequency-update loop inside the NAP; both methods remain well within interactive time on this instance.

Run	SA z	gap%	GA z	gap%	Run	SA z	gap%	GA z	gap%
0	45865.1	1.11	50575.9	11.5	10	46523.8	2.56	50769.7	11.9
1	49779.1	9.74	45855.7	1.1	11	51738.4	14.06	50385.1	11.1
2	51410.6	13.34	53768.3	18.5	12	45361.1	0.00	49570.6	9.3
3	47418.5	4.54	51146.1	12.8	13	50817.7	12.03	53988.3	19.0
4	48962.2	7.94	54969.3	21.2	14	46783.4	3.13	51201.8	12.9
5	48159.4	6.17	53423.1	17.8	15	46411.3	2.31	52050.2	14.7
6	51364.6	13.23	58148.6	28.2	16	45520.6	0.35	56075.9	23.6
7	51089.7	12.63	51763.1	14.1	17	49364.1	8.82	57414.9	26.6
8	50580.4	11.51	50563.4	11.5	18	50944.5	12.31	54944.0	21.1
9	45749.9	0.86	56059.5	23.6	19	52989.6	16.82	56139.6	23.8

Table 4: Per-run objective values and gaps to the best-known solution ($z^* = 45361.1$) for all 20 replications of each method.

Figure 1 shows the convergence of the incumbent objective for the best SA run. The incumbent drops rapidly from a heavily penalised infeasible start while the temperature is high, then refines locally as $T \rightarrow 0$, finishing at the feasible optimum $z = 45361.1$. Figure 2 contrasts the objective distributions of the two methods over the 20 runs.

4.4 Best Solution Found

The best feasible solution comprises four routes, with the endogenous headways and load factors shown in Table 5. It serves 87.95% of demand (unsatisfied fraction $12.05\% < U_{\max} = 15\%$), uses 39.2 of 58 available vehicles,

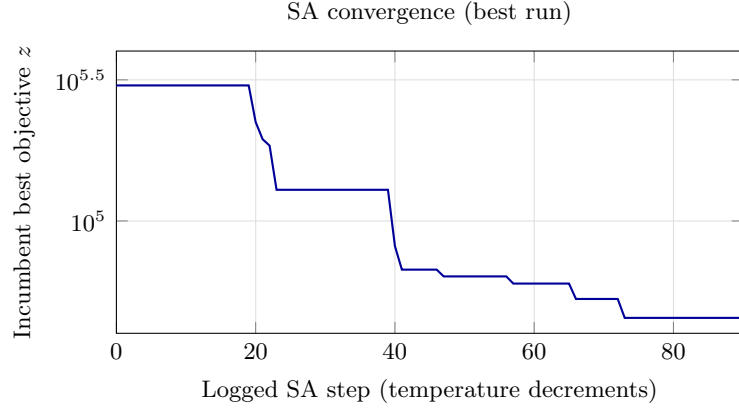


Figure 1: Convergence of the incumbent objective for the best SA run (logarithmic vertical axis).

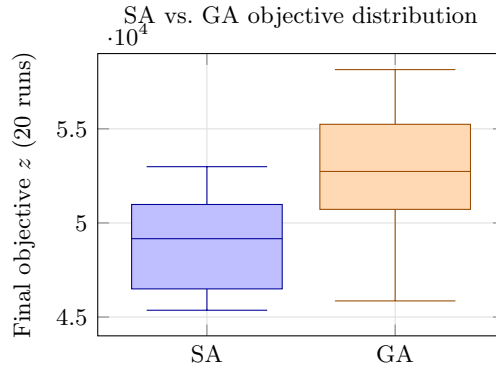


Figure 2: Distribution of the final objective over 20 runs. SA (left) is both lower and tighter than GA (right).

and every route operates exactly at the load-factor ceiling $L_{\max} = 1.00$, indicating an efficient, well-utilised network. The objective decomposes into user cost 99285.3, operator cost 9797.5, and unsatisfied-demand cost 8640.0, giving the weighted total $z = 45361.1$.

Route	Node sequence	Length (km)	Headway (min)	Load factor
R_1	3-6-9-8-11-12-14	14.7	7.3	1.00
R_2	6-9-8-10-12-13-15	15.5	5.2	1.00
R_3	6-4-7-8-11-9	13.8	6.0	1.00
R_4	1-3-4-7-10-12-14	15.6	5.8	1.00

Table 5: Best feasible solution found ($z = 45361.1$).

4.5 Statistical Analysis

To test whether SA significantly outperforms the GA, three tests were applied to the 20-run objective samples: a Welch two-sample t -test (unequal variances), a Mann-Whitney U test (rank-based, distribution-free), and a Wilcoxon signed-rank test on the run-matched differences. The results are in Table 6.

Test	Statistic	p -value	Conclusion
Welch t -test	-4.650	4.28×10^{-5}	SA < GA
Mann-Whitney U	67.0	3.38×10^{-4}	SA < GA
Wilcoxon signed-rank	12.0	1.34×10^{-4}	SA < GA

Table 6: Statistical comparison of SA versus GA objective values (20 runs each).

All three tests reject the null hypothesis of equal performance at the 1% level, with $p < 10^{-3}$ in every case. The parametric and non-parametric tests agree, so the conclusion is robust to distributional assumptions: SA produces significantly lower objective values than the GA on this instance, in agreement with the comparative findings of [1].

5 Source Code Repository

The complete source code, the generated benchmark data, the experimental result files, and a README explaining how to run each mode are available in the public repository:

<https://github.com/FAVURAL/676>

The repository contains `btrndp.sa.py` (the solver with the SA and GA algorithms, the experiment drivers, and the statistical tests), `generate_instance.py` (the deterministic instance generator), `requirements.txt`, the `data` directory (CSV inputs), and the `results` directory (CSV outputs and figures). The README documents the installation requirements (Python 3.9+, `numpy`, `scipy`) and the command-line modes used to reproduce every number in this report.

6 Conclusions

This project delivered a complete, working implementation of the Fan and Machemehl [1] Simulated Annealing framework for the BTRNDP, comprising an ICRSGP candidate generator (Dijkstra and Yen), a lexicographic NAP evaluator with endogenous frequency setting, and an SA optimiser with a penalty-based feasibility mechanism, together with a GA baseline. On a deterministically generated 15-node benchmark, SA attained a best feasible objective of $z = 45361.1$ with a mean gap of 7.67% and significantly outperformed the GA across three statistical tests ($p < 10^{-3}$). Parameter tuning reproduced the optimal region reported in the literature ($T_0 = 2000$, $\alpha = 0.6$).

The principal modelling choices made for reproducibility – the self-generated benchmark, the bus capacity $P = 90$, and the use of the best-found value as the gap reference – are stated explicitly so that the results can be interpreted correctly. Future work includes testing on additional and larger networks, adding an explicit transfer-flow conservation model, and hybridising SA with a local search to accelerate late-stage convergence.

References

- [1] Fan, W. and Machemehl, R. B.,
Using a Simulated Annealing Algorithm to Solve the Transit Route Network Design Problem.
Journal of Transportation Engineering, Vol. 132, No. 2, pp. 122–132, 2006.
- [2] Ahuja, R. K., Magnanti, T. L. and Orlin, J. B.,
Network Flows: Theory, Algorithms, and Applications.
Prentice–Hall, Englewood Cliffs, NJ, 1993. ISBN 0-13-617549-X.
- [3] Yen, J. Y.,
Finding the K Shortest Loopless Paths in a Network.
Management Science, Vol. 17, No. 11, pp. 712–716, 1971.
- [4] Eglese, R. W.,
Simulated Annealing: A Tool for Operational Research.
European Journal of Operational Research, Vol. 46, pp. 271–281, 1990.
- [5] Pattnaik, S. B., Mohan, S. and Tom, V. M.,
Urban Bus Transit Network Design Using Genetic Algorithm.
Journal of Transportation Engineering, Vol. 124, No. 4, pp. 368–375, 1998.